

hw7
my uni:sn3136

1.

register numbers:

$$\$t3 = 11, \quad \$s0 = 16, \quad \$s1 = 17, \quad \$s2 = 18, \quad \$s3 = 19.$$

(a) **lw \$t3, 40(\$s2)**

I-type format: `op | rs | rt | immediate`

$$\text{op} = 100011, \quad rs = 10010, \quad rt = 01011, \quad \text{imm} = 0000000000101000.$$

so:

100011 10010 01011 0000000000101000

hex:

0x8E4B0028

(b) **add \$t3, \$s2, \$s1**

R-type format: `op | rs | rt | rd | shamt | funct`

$$\text{op} = 000000, \quad rs = 10010, \quad rt = 10001, \quad rd = 01011, \quad \text{shamt} = 00000, \quad \text{funct} = 100000.$$

so:

000000 10010 10001 01011 00000 100000

hex:

0x02515820

(c) **addi \$t3, \$s0, 37**

I-type format: `op | rs | rt | immediate`

$$\text{op} = 001000, \quad rs = 10000, \quad rt = 01011, \quad \text{imm} = 0000000000100101.$$

then:

001000 10000 01011 0000000000100101

hex:

0x220B0025

(d) `beq $s1, $s3, LABEL`

I-type format: `op | rs | rt | immediate`. since the label is 7 instructions before the current `beq` and the branch offset is relative to the next instruction, so the immediate is

$$-7 - 1 = -8.$$

In 16-bit two's complement, -8 is

11111111111111000.

so:

000100 | 10001 | 10011 | 1111111111111000

hex:

0x1233FFF8

(e)

J-type format: `op | target`. The jump target field stores the word address:

$$2088/4 = 522.$$

now 522 in 26-bit binary is

0000000000000000001000001010.

so:

000010 | 00000000000000001000001010

hex:

0x0800020A

2.

they're separate so that one opcode can identify a broad instruction class while the function field selects the exact ALU operation within that class which uses encoding space more efficiently and lets many R-type instructions share the same opcode

3.

for each case, we list which instruction types fail and what goes wrong

(a) RegWrite = 0

- **R-type:** fails, the ALU result is computed but never written to the register file.
- **I-type:** fails, the ALU/immediate result is not written back.
- **beq:** works, branches do not write a register.

- **lw:** fails, the memory value is loaded but never written into the destination register.
- **sw:** works, stores do not write a register.

(b) **ALUOp0 = 0**

Normally **beq** uses **ALUOp=01** so the ALU subtracts the two register operands to test equality but if **ALUOp0** is stuck at 0, then **beq** instead gets the wrong ALU control

- **R-type:** works
- **I-type:** works
- **beq:** fails, the ALU no longer performs the correct subtraction for equality testing, so the branch decision can be wrong
- **lw:** works
- **sw:** works

(c)

this forces instruction types that normally use **ALUOp=00** or **01** to be interpreted as if they need R-type-style ALU decoding, which is wrong for their actual operation

- **R-type:** works, it already expects **ALUOp=10**
- **I-type:** fails, the ALU operation is decoded incorrectly instead of doing the intended immediate arithmetic
- **beq:** fails, the equality comparison uses the wrong ALU operation, so the branch decision can be wrong
- **lw:** fails, the effective address computation is wrong
- **sw:** fails, the effective address computation is wrong

(d)

ow the branch selection logic is always enabled so any non-**beq** instruction may branch accidentally whenever the ALU's Zero output happens to be 1

- **R-type:** can fail by changing control flow unexpectedly when the ALU result is zero
- **I-type:** can fail by branching unexpectedly when the ALU result is zero
- **beq:** works as intended
- **lw:** can fail by branching unexpectedly, for example if the computed address is zero
- **sw:** can fail by branching unexpectedly, for example if the computed address is zero

(e) **MemRead** = 0

- **R-type:** works
- **I-type:** works
- **beq:** works
- **lw:** fails, memory is never read, so the loaded value is wrong or unavailable
- **sw:** works

(f) **MemWrite** = 0

- **R-type:** works
- **I-type:** works
- **beq:** works
- **lw:** works
- **sw:** fails, the store never updates memory

(g) **RegDest** = 1

this forces the destination register to come from the **rd** field instead of the **rt** field

- **R-type:** works, R-type instructions already write to **rd**
- **I-type:** fails, it writes the result to the wrong register
- **beq:** works
- **lw:** fails, it writes the loaded value to the wrong register
- **sw:** works

(h) **MemToReg** = 0

this forces register writeback to come from the ALU result instead of the data memory output

- **R-type:** works, R-type instructions already write back the ALU result
- **I-type:** works, immediate arithmetic also writes back the ALU result
- **beq:** works
- **lw:** fails, it writes the computed address rather than the memory value
- **sw:** works

(i) **ALUSrc** = 0

this forces the ALU's second input to come from the second register output rather than from the sign-extended immediate

- **R-type**: works R-type instructions already use two register operands
- **I-type**: fails, it uses the wrong second operand instead of the immediate
- **beq**: works, **beq** compares two registers
- **lw**: fails, it computes the wrong address.
- **sw**: fails, it computes the wrong address.

4.

we're are told to add a 2-bit control signal **BranchType**, with

$$00 = \text{beq}, \quad 01 = \text{bne}, \quad 10 = \text{bltz}, \quad 11 = \text{bgez}$$

we're also told that for **bltz** and **bgez**, the second input register is forced to **\$zero** so the ALU computes

$$rs - 0 = rs,$$

so the ALU's **Negative** flag directly indicates whether the register value is negative

Required datapath changes

1. add a mux on the second register-number input to the register file so that the second source register can be either the instruction's usual **rt** field or register 0 (that is, **\$zero**), we choose **\$zero** for **bltz** and **bgez**
2. Keep the ALU's existing **Zero** flag and add the new **Negative** flag output
3. Replace the simple branch decision logic with logic that depends on **BranchType**, **Zero**, and **Negative**

Branch-decision logic

so we let **BranchType** be B_1B_0 , let Z be the Zero flag, and let N be the Negative flag and then the branch should be taken exactly when

$$(\overline{B_1} \overline{B_0} Z) \vee (\overline{B_1} B_0 \overline{Z}) \vee (B_1 \overline{B_0} N) \vee (B_1 B_0 \overline{N})$$

this corresponds to:

- **beq**: branch when $Z = 1$,

- **bne**: branch when $Z = 0$,
- **bltz**: branch when $N = 1$,
- **bgez**: branch when $N = 0$

so the output of this logic replaces the old single **Branch AND Zero** condition that controlled the PC-select mux

5.

(a) Modified **sw**

so:

sw \$s0, \$s1, 13

means:

- use the contents of **\$s1** as the memory address,
- compute $(\$s0) + 13$,
- store that sum into memory at address $(\$s1)$

so the architecture must be changed so that:

1. the memory *address* can come directly from the base register value rather than from the ALU result,
2. the ALU can add the source register value and the immediate,
3. the memory *write-data* input can come from the ALU result rather than directly from the register file

a clean way to do this is to add:

- a mux before the memory address input: choose either **ALUResult** or the register value used as the address,
- a mux before the memory write-data input: choose either the normal register-file data output or **ALUResult**,
- a mux before ALU input A: choose which register value is to be combined with the immediate

(b) Modified `lw`

so:

`lw $s0, $s1, 13`

means:

- use the contents of `$s1` as the memory address,
- read memory at that address,
- add 13 to the value read from memory,
- write the result into `$s0`

so the architecture must be changed so that:

1. memory can be addressed directly by the base register value,
2. the value read from memory can be routed to an ALU input,
3. the ALU can add that memory value to the immediate,
4. the ALU result can be written back to the destination register

a clean implementation is to add:

- a mux before the memory address input,
- a mux before ALU input A so it can choose memory output,
- the usual ALU immediate path for ALU input B,
- the normal register writeback path from the ALU result

(c)

both instructions can be supported with a single ALU by using muxes to choose whether the ALU comes *before* or *after* memory access

- for the modified `sw`, the order is:

register value $\longrightarrow$ ALU add immediate $\longrightarrow$ memory write-data

memory address comes directly from the address register

- for the modified `lw`, the order is:

memory read $\longrightarrow$ ALU add immediate $\longrightarrow$ register writeback

so basically, the memory address comes directly from the address register

so with one ALU, the trick is to add muxes that let the datapath choose:

1. whether memory address comes from the ALU result or directly from a register,
2. whether ALU input A comes from a register or from memory output,
3. whether memory write-data comes from a register or from the ALU result

those muxes allow the datapath to perform either *ALU then memory* (new **sw**) or *memory then ALU* (new **lw**) while still using only one ALU